19

分页：更快的转换（TLB）

使用分页（paging）作为支持虚拟内存（virtual memory）的核心机制，可能会带来很高的性能开销。分页把地址空间（address space）切成很小的、大小固定的单元，也就是页（page），因此需要大量的映射信息。由于这些映射信息一般存放在物理内存（physical memory）中，从逻辑上讲，程序每产生一个虚拟地址（virtual address），分页都需要额外查一次内存。每次取指令或者显式加载、存储之前，都先去内存里取转换信息，这样做慢得令人无法接受。于是，我们的问题是：

关键问题：
如何加速地址转换
如何加速地址转换（address translation），尽量避免分页似乎必不可少的那次额外内存引用？需要什么样的硬件支持？操作系统需要怎样参与？

想让事情变快时，操作系统通常需要一些帮助。而帮助往往来自操作系统的老朋友：硬件。为了加速地址转换，我们要加入一个叫作**转换后备缓冲区**（translation-lookaside buffer，**TLB**，也常称快表）的部件 [CG68, C95]，之所以叫这个名字，是出于历史原因 [CP78]。TLB 是芯片上**内存管理单元**（memory-management unit，**MMU**）的一部分，它就是一个硬件**缓存**（cache），存放常用的虚拟地址到物理地址（physical address）的转换；所以，叫它**地址转换缓存**（address-translation cache）会更贴切。每次虚拟内存引用时，硬件都先检查 TLB，看里面有没有所需的转换；如果有，就（快速地）完成转换，无须查询存有全部转换的页表（page table）。TLB 对性能的影响极大，可以说，正是 TLB 让虚拟内存真正变得可行 [C95]。

19.1 TLB 的基本算法

图 19.1 粗略描述了硬件可能怎样处理虚拟地址转换，这里假设页表是一个简单的数组，即**线性页表**（linear page table），并且使用**硬件管理的 TLB**（hardware-managed TLB），即访问页表的大部分工作由硬件承担（下面会进一步解释）。

硬件遵循的算法是这样的：首先，从虚拟地址中提取虚拟页号（virtual page number，VPN）（图 19.1 的第 1 行），然后检查 TLB 中是否有这个 VPN 的转换（第 2 行）。如果有，就是一次 **TLB 命中**（TLB hit），说明 TLB 中存有这个转换。成功了！接下来，我们就可以

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)    // TLB 命中
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset   = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else                    // TLB 未命中
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else if (CanAccess(PTE.ProtectBits) == False)
16         RaiseException(PROTECTION_FAULT)
17     else
18         TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
19         RetryInstruction()
```

图 19.1：**TLB 控制流算法**

从相应的 TLB 项中取出页帧号（PFN），把它和原虚拟地址中的偏移量（offset）拼接起来，得到所需的物理地址，然后访问内存（第 5–7 行），前提是保护检查没有失败（第 4 行）。

如果 CPU 在 TLB 中没有找到转换，即发生 **TLB 未命中**（TLB miss），就还有一些工作要做。在这个例子中，硬件会访问页表来查找转换（第 11–12 行）；如果进程（process）产生的虚拟内存引用有效且可以访问（第 13、15 行），就用这个转换更新 TLB（第 18 行）。这一系列操作代价高昂，主要是因为访问页表需要一次额外的内存引用（第 12 行）。最后，TLB 更新之后，硬件会重新执行这条指令；这一次，转换可以在 TLB 中找到，内存引用很快就能完成。

和所有缓存一样，TLB 也建立在这样一个前提上：在通常情况下，转换都能在缓存中找到（也就是命中）。如果命中，增加的开销就很小，因为 TLB 就在处理器核心附近，而且设计得非常快。如果未命中，就要付出分页的高昂代价：必须访问页表来查找转换，从而带来一次额外的内存引用（页表更复杂的话还会更多）。如果这种情况经常发生，程序很可能会明显变慢；与大多数 CPU 指令相比，内存访问的代价相当高，而 TLB 未命中会导致更多的内存访问。因此，我们希望尽可能避免 TLB 未命中。

19.2 示例：访问数组

为了把 TLB 的工作方式讲清楚，我们来看一个简单的虚拟地址跟踪，看看 TLB 怎样提升它的性能。在这个例子中，假设内存中有一个由 10 个 4 字节整数组成的数组，起始虚拟地址是 100。再假设我们有一个很小的 8 位虚拟地址空间（virtual address space），页大小（page size）为 16 字节；这样，虚拟地址就分成 4 位的 VPN 和 4 位的偏移量：共有 16 个虚拟页（virtual page），每页 16 个字节。

图 19.2（第 3 页）展示了数组在系统的 16 个 16 字节页上的布局。可以看到，数组的第一个元素（a[0]）从 VPN=06、偏移量=04 处开始；那一页只能放下三个 4 字节整数。数组接着延伸到下一页（VPN=07），接下来的四个元素（a[3] … a[6]）都在这一页上。这个数组的最后三个元素（a[7] … a[9]）则位于地址空间的再下一页（VPN=08）。

现在考虑一个依次访问数组每个元素的简单循环，用 C 语言写出来大致是这样的：

```
int i, sum = 0;
for (i = 0; i < 10; i++) {
    sum += a[i];
}
```

为简单起见，我们姑且假定这个循环只访问数组的内存（忽略变量 i 和 sum，以及指令本身）。访问数组的第一个元素（a[0]）时，CPU 会看到一次对虚拟地址 100 的加载。硬件从中提取出 VPN（VPN=06），用它在 TLB 中查找有效的转换。假设这是程序第一次访问这个数组，结果会是一次 TLB 未命中。

下一次访问的是 a[1]，这里有个好消息：TLB 命中！数组的第二个元素紧挨着第一个元素，和它在同一页上；访问第一个元素时已经访问过这一页，转换已经加载到 TLB 中了，所以这次能成功。访问 a[2] 同样成功（又一次命中），因为它也和 a[0]、a[1] 位于同一页。

遗憾的是，程序访问 a[3] 时，又遇到了一次 TLB 未命中。不过，接下来的几个元素（a[4] … a[6]）又都会在 TLB 中命中，因为它们都位于内存中的同一页。

最后，访问 a[7] 又引发了一次 TLB 未命中，这也是最后一次。硬件再次查询页表，弄清这个虚拟页在物理内存中的位置，并相应地更新 TLB。剩下的两次访问（a[8] 和 a[9]）都受益于这次 TLB 更新：硬件在 TLB 中查找它们的转换时，又得到两次命中。

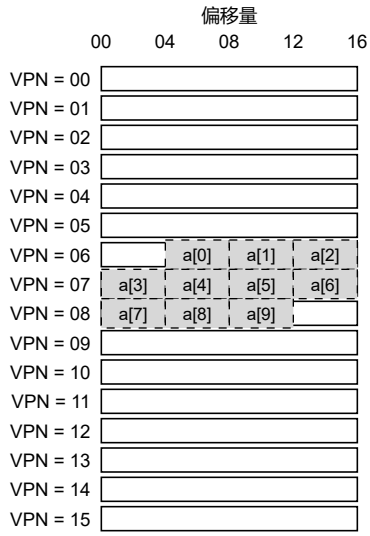


图 19.2：**示例：一个很小的地址空间中的数组**

提示：尽可能使用缓存

缓存是计算机系统中最基本的性能技术之一，人们一再用它来“让常见情况（common case）更快”[HP06]。硬件缓存背后的思想是利用指令和数据引用中的**局部性**（locality）。局部性通常有两种：**时间局部性**（temporal locality）和**空间局部性**（spatial locality）。时间局部性是指，最近访问过的指令或数据项，很可能很快会被再次访问。想想循环变量或循环中的指令：它们会在一段时间内被反复访问。空间局部性是指，如果程序访问了地址 x 处的内存，它很可能很快就会访问 x 附近的内存。想象一下顺序遍历某个数组，访问完一个元素再访问下一个。当然，这些性质取决于程序的具体特点，因此它们并不是铁律，更像是经验法则。

无论是用于指令、数据还是地址转换（比如我们的 TLB），硬件缓存利用局部性的办法都是把内存中的内容复制一份，放在容量小、速度快的片上存储器里。处理器不必非得去（缓慢的）内存里满足请求，而是可以先看看缓存中有没有就近的副本；如果有，处理器就能快速访问它（也就是只需几个 CPU 周期），避免花费访问内存所需的大量时间（许多纳秒）。

你可能会想：既然 TLB 这类缓存这么好，为什么不干脆把缓存做得更大，把所有数据都放进去呢？遗憾的是，这就会碰上物理定律这类更根本的限制。缓存要快，就必须小，因为光速之类的物理限制会起作用。大缓存注定是慢的，这就失去了缓存的意义。因此，我们只能用小而快的缓存；剩下的问题是，如何最好地利用它们来提升性能。

我们来总结一下这十次数组访问期间 TLB 的活动：**未命中**、命中、命中、**未命中**、命中、命中、命中、**未命中**、命中、命中。因此，TLB 的**命中率**（hit rate，即命中次数除以总访问次数）是 70%。我们其实希望命中率接近 100%，所以这个数字不算高，但它不是零，这可能有点出人意料。尽管这是程序第一次访问这个数组，得益于**空间局部性**（spatial locality），TLB 还是提升了性能。数组的元素紧密地排列在页中（也就是说，它们在**空间**上彼此靠近），因此只有第一次访问某一页上的元素时，才会出现 TLB 未命中。

还要注意页大小在这个例子中起的作用。哪怕页大小只是大一倍（32 字节而不是 16 字节），访问数组时的未命中次数也会更少。典型的页大小更接近 4 KB，因此这类密集的、基于数组的访问能获得极佳的 TLB 性能，每一页只会遇到一次未命中。

关于 TLB 的性能还有最后一点：如果程序在这个循环结束后不久再次访问这个数组，假设 TLB 足够大，能缓存所需的转换，那么我们很可能会看到更好的结果：命中、命中、命中、命中、命中、命中、命中、命中、命中、命中。在这种情况下，TLB 命中率（TLB hit rate）之所以高，是因为**时间局部性**（temporal locality），也就是内存项在**时间**上很快被再次引用。和任何缓存一样，TLB 的成功依赖于空间局部性和时间局部性，而这两者都是程序的性质。如果所关心的程序具有这样的局部性，TLB 命中率（TLB hit rate）就很可能很高；很多程序都是如此。

 © 2008–23, ARPACI-DUSSEAU

```
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True)    // TLB 命中
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset   = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else                    // TLB 未命中
11     RaiseException(TLB_MISS)
```

图 19.3：**TLB 控制流算法（由操作系统处理）**

19.3 谁来处理 TLB 未命中?

我们必须回答一个问题：谁来处理 TLB 未命中?可能的答案有两个：硬件，或者软件（操作系统）。早年间，硬件的指令集很复杂（有时称为 **CISC**，即复杂指令集计算机，complex-instruction set computer），而造硬件的人不太信任那些狡猾的操作系统开发者。因此，TLB 未命中完全交给硬件处理。为此，硬件必须确切地知道页表在内存中的位置（通过**页表基址寄存器**，page-table base register，图 19.1 的第 11 行就用到了它），以及页表的确切格式；发生未命中时，硬件会“遍历”页表，找到正确的页表项（page table entry）并取出所需的转换，用这个转换更新 TLB，然后重新执行指令。Intel x86 就是采用**硬件管理 TLB**（hardware-managed TLB）的“较老”架构之一：它使用固定的**多级页表**（multi-level page table，详见下一章），当前页表由 CR3 寄存器指向 [I09]。

更现代的架构（例如 MIPS R10k [H93] 或 Sun 的 SPARC v9 [WG00]，它们都是 **RISC**，即精简指令集计算机，reduced-instruction set computer）采用所谓的**软件管理的 TLB**（software-managed TLB）。发生 TLB 未命中时，硬件只是触发一个异常（图 19.3 的第 11 行），暂停当前的指令流，把特权级提升到内核模式（kernel mode），然后跳转到**陷阱处理程序**（trap handler）。你可能已经猜到，这个陷阱处理程序（trap handler）就是操作系统中专门为处理 TLB 未命中而编写的代码。这段代码运行时，会在页表中查找转换，用特殊的“特权”指令更新 TLB，然后从陷阱返回（return-from-trap）；此时，硬件会重新执行这条指令，这次会 TLB 命中。

我们来讨论几个重要的细节。第一，这里的从陷阱返回（return-from-trap）指令，要和之前处理系统调用（system call）时看到的从陷阱返回指令稍有不同。在系统调用的情况下，从陷阱返回（return-from-trap）应该从陷入操作系统那条指令的下一条指令处继续执行，就像从过程调用（procedure call）返回时，会回到紧跟在调用指令之后的那条指令一样。而在 TLB 未命中的情况下，从未命中处理的陷阱（trap）返回时，硬件必须从引发陷阱的那条指令处继续执行；这样重试（retry）一次，指令就会再运行一遍，这次会 TLB 命中。因此，根据陷阱（trap）或异常的引发方式，硬件在陷入操作系统时必须保存不同的 PC，这样到了该恢复的时候才能正确地继续执行。

补充：RISC 与 CISC

20 世纪 80 年代，计算机体系结构界发生了一场大论战。一方是 **CISC** 阵营，CISC 是**复杂指令集计算**（Complex Instruction Set Computing）的缩写；另一方是 **RISC**，即**精简指令集计算**（Reduced Instruction Set Computing）[PS81]。RISC 一方的领军人物是伯克利的 David Patterson 和斯坦福的 John Hennessy（两人也合写过几本著名的书 [HP06]），不过，后来因 RISC 方面的开创性工作而获得图灵奖（Turing Award）的是 John Cocke [CM00]。

CISC 指令集往往包含大量指令，而且每条指令的功能都相对强大。例如，你可能会看到一条字符串复制指令，它接受两个指针和一个长度，把字节从源地址复制到目标地址。CISC 背后的思想是，指令应该是高级原语，这样汇编语言本身更容易使用，代码也更紧凑。

RISC 指令集恰恰相反。RISC 背后的一个关键观察是：指令集其实是编译器的生成目标，而编译器真正想要的只是几个简单的原语，用来生成高性能的代码。因此，RISC 的支持者主张，尽可能多地把东西从硬件中去掉（尤其是微码），让剩下的部分简单、统一、快速。

早期，RISC 芯片影响巨大，因为它们明显更快 [BC91]；人们写了许多论文，还成立了几家公司（例如 MIPS 和 Sun）。然而，随着时间推移，Intel 等 CISC 厂商把许多 RISC 技术融入了处理器核心，例如在流水线前端增加几级，把复杂指令转换成微指令，再以类似 RISC 的方式处理。这些创新，再加上每块芯片上的晶体管越来越多，让 CISC 保持了竞争力。最终，这场争论平息了，如今两类处理器都能做得很快。

第二，运行 TLB 未命中处理代码时，操作系统要格外小心，不能引发无穷无尽的 TLB 未命中。解决办法有很多；例如，可以把 TLB 未命中处理程序放在物理内存中，这部分内存**不经映射**（unmapped），也不需要地址转换；或者在 TLB 中保留一些项用于永久有效的转换，并把其中一些永久转换的槽位留给处理程序代码本身；这些**固定**（wired）转换总能在 TLB 中命中。

软件管理方法的主要优点是*灵活性*：操作系统可以用任何它想用的数据结构（data structure）来实现页表，而不必改动硬件。另一个优点是*简单*，这一点从 TLB 控制流就能看出来（对比图 19.3 中的第 11 行和图 19.1 中的第 11–19 行）。发生未命中时，硬件不用做太多事：只需触发一个异常，剩下的交给操作系统的 TLB 未命中处理程序即可。

19.4 TLB 的内容：里面有什么?

我们来更详细地看看硬件 TLB 的内容。典型的 TLB 可能有 32、64 或 128 项，并且采用所谓的**全相联**（fully associative）结构。简单来说，这意味着任何一个转换都可以放在 TLB 中的任何位置，硬件会并行搜索整个 TLB 来找到所需的转换。一个 TLB 项大概是这样的：

VPN | PFN | 其他位

补充：TLB 的有效位 ≠ 页表的有效位

一个常见的错误是把 TLB 中的有效位（valid bit）和页表中的有效位混为一谈。在页表中，如果某个页表项（PTE）被标记为无效，就表示进程还没有分配这个页，正常工作的程序不应该访问它。访问无效页时，通常的反应是陷入操作系统，由操作系统杀死这个进程。

相比之下，TLB 的有效位只是表示某个 TLB 项中是否存有有效的转换。例如，系统启动时，每个 TLB 项通常的初始状态都是无效，因为那里还没有缓存任何地址转换。一旦启用了虚拟内存，程序开始运行并访问各自的虚拟地址空间，TLB 就会逐渐被填充，很快就满是有效项了。

TLB 的有效位在执行上下文切换（context switch）时也很有用，下面会进一步讨论。通过把所有 TLB 项都设为无效，系统可以确保即将运行的进程不会误用前一个进程的虚拟地址到物理地址的转换。

注意，每一项中都同时有 VPN 和 PFN，因为一个转换可能被放在任何一个位置（用硬件术语来说，TLB 是一种**全相联缓存**，fully-associative cache）。硬件会并行搜索这些项，看有没有匹配的。

更有意思的是“其他位”。例如，TLB 通常有一个**有效**（valid）位，表示这一项中是否存有有效的转换。另一类常见的是**保护**（protection）位，决定一个页可以怎样被访问，这和页表中一样。例如，代码页（code page）可能被标记为可读、可执行，而堆页（heap page）可能被标记为可读、可写。此外，可能还有其他几个字段，包括**地址空间标识符**（address-space identifier）、**脏位**（dirty bit）等等；更多信息见下文。

19.5 TLB 的问题：上下文切换

有了 TLB，在进程之间，也就是在地址空间之间切换时，就会出现新问题。具体来说，TLB 中存放的虚拟地址到物理地址的转换，只对当前运行的进程有效；这些转换对其他进程没有意义。因此，从一个进程切换到另一个进程时，硬件或操作系统（或两者一起）必须小心，确保即将运行的进程不会误用之前运行的某个进程的转换。

为了更好地理解这种情况，我们来看一个例子。一个进程 P1 在运行时，会假定 TLB 中可能缓存着对它有效的转换，也就是来自 P1 页表的转换。在这个例子中，假设 P1 的第 10 个虚拟页映射到物理帧（physical frame）100。

再假设还有另一个进程 P2，操作系统可能很快就会决定进行上下文切换，转而运行它。假设 P2 的第 10 个虚拟页映射到物理帧 170。如果两个进程的项都在 TLB 中，TLB 的内容就会是这样：

VPN	PFN	有效	保护
10	100	1	rwx
—	—	0	—
10	170	1	rwx
—	—	0	—

在上面的 TLB 中，问题显而易见：VPN 10 既可能转换成 PFN 100（P1），也可能转换成 PFN 170（P2），但硬件无法分辨哪一项属于哪个进程。因此，要让 TLB 正确、高效地支持多个进程的虚拟化（virtualization），我们还得多做些工作。于是，关键问题来了：

关键问题：
上下文切换时如何管理 TLB 的内容
在进程之间进行上下文切换时，TLB 中上一个进程的转换对即将运行的进程没有意义。为了解决这个问题，硬件或操作系统应该怎么做？

这个问题有许多可能的解决办法。一种方法是在上下文切换时直接**刷新**（flush）TLB，在运行下一个进程之前把它清空。在基于软件的系统中，可以用一条显式的（特权）硬件指令来完成刷新；在硬件管理 TLB 的系统中，可以在页表基址寄存器（page-table base register）改变时执行刷新；注意，操作系统在上下文切换时反正都要修改 PTBR。无论哪种情况，刷新操作都只是把所有有效位设为 0，本质上就是清空 TLB 的内容。

在每次上下文切换时刷新 TLB，就得到了一个可行的方案，因为进程永远不会意外地用到 TLB 中错误的转换。然而，这是有代价的：进程每次运行时，访问自己的数据页（data page）和代码页都必然会遇到 TLB 未命中。如果操作系统频繁地在进程之间切换，这个代价可能会很高。

为了降低这种开销，有些系统增加了硬件支持，让 TLB 可以跨上下文切换共享。具体来说，有些硬件系统在 TLB 中提供了**地址空间标识符**（address space identifier，**ASID**）字段。可以把 ASID 看作**进程标识符**（process identifier，**PID**），只是它的位数通常更少（例如 ASID 是 8 位，而 PID 是 32 位）。

如果给上面那个 TLB 示例加上 ASID，显然进程就能很方便地共享 TLB 了：只需 ASID 字段，就能区分原本完全相同的转换。下面是加上 ASID 字段后的 TLB：

VPN	PFN	有效	保护	ASID
10	100	1	rwx	1
—	—	0	—	—
10	170	1	rwx	2
—	—	0	—	—

因此，有了地址空间标识符（address space identifier），TLB 就能同时存放来自不同进程的转换，而不会混淆。当然，硬件还需要知道当前运行的是哪个进程，才能执行转换，所以操作系统在上下文切换时，必须把某个特权寄存器设为当前进程的 ASID。

顺便说一下，你可能还想到了另一种情况：TLB 中有两项非常相似。在下面这个例子中，两个不同进程的两项，VPN 不同，却指向同一个物理页（physical page）：

VPN	PFN	有效	保护	ASID
10	101	1	r-x	1
—	—	0	—	—
50	101	1	r-x	2

 © 2008–23, ARPACI-DUSSEAU

–	–	0	–	–

例如，当两个进程*共享*一个页，比如代码页时，就可能出现这种情况。在上面的例子中，进程 1 和进程 2 共享物理页 101；P1 把这个页映射到它地址空间的第 10 页，而 P2 把它映射到自己地址空间的第 50 页。在二进制程序或共享库中共享代码页很有用，因为它可以减少使用中的物理页数量，从而降低内存开销。

19.6 问题：替换策略

和任何缓存一样，TLB 还需要考虑另一个问题：**缓存替换**（cache replacement）。具体来说，往 TLB 中放入新项时，必须**替换**（replace）掉一个旧项，于是问题来了：替换哪一个？

> 关键问题：如何设计 TLB 替换策略
> 往 TLB 中加入新项时，应该替换哪一项？当然，目标是让**未命中率**（miss rate）尽可能低，或者说让**命中率**（hit rate）尽可能高，从而提升性能。

等到研究把页交换到磁盘的问题时，我们会比较详细地讨论这类策略；这里只简单介绍几种典型的策略。一种常见的方法是淘汰（evict）**最近最少使用**（least-recently-used，**LRU**）的项。LRU 试图利用内存引用流中的局部性，它假设最近没有被使用过的项很可能就是合适的淘汰对象。另一种典型的方法是采用**随机**（random）策略，随机淘汰一个 TLB 映射。这种策略简单，而且能避免一些极端情况下的反常表现，因而很有用；例如，当程序在大小为 n 的 TLB 上循环访问 $n+1$ 个页时，像 LRU 这样“合理”的策略就会表现得极不合理：LRU 每次访问都会未命中，而随机策略要好得多。

19.7 一个真实的 TLB 项

最后，我们简单看一个真实的 TLB。这个例子来自 MIPS R4000 [H93]，它是一个使用软件管理 TLB 的现代系统；图 19.4 展示了一个稍作简化的 MIPS TLB 项。

MIPS R4000 支持 32 位地址空间，页大小为 4 KB。因此可以预料，典型的虚拟地址会分成 20 位的 VPN 和 12 位的偏移量。然而，从图中的 TLB 项可以看到，VPN 只有 19 位；原来，用户地址只会来自地址空间的一半（另一半留给内核），因此只需要 19 位的 VPN。VPN 可以转换成最多 24 位的物理帧号（PFN），因此可以支持最多 64 GB（物理）主存的系统（2^{24} 个 4 KB 的页）。

MIPS TLB 中还有其他几个有意思的位。其中有一个*全局*位（global，G），用于在进程之间全局共享的页；如果设置了全局位，ASID 就会被忽略。还有 8 位的 *ASID*，操作系统可以用它来区分不同的地址空间，这在前面讲过。给你提个问题：如果同时运行的进程超过 256（2^8）个，操作系统该怎么办？此外还有 3 个*一致性*（Coherence，C）位，决定硬件如何缓存这个页，这有点超出本书的范围；一个*脏*位，页被写过之后置位（后面会看到它的用途）；一个*有效*位，告诉硬件这一项中是否存有有效的转换。另外还有一个*页掩码*（page

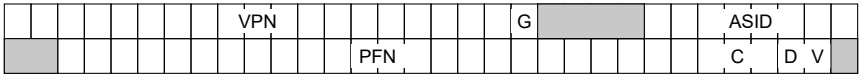


图 19.4：**一个 MIPS TLB 项**

提示：RAM 不总是 RAM（Culler 定律）

随机存取存储器（random-access memory，**RAM**）这个名字意味着，访问 RAM 的任何部分都一样快。一般来说，这样看待 RAM 没什么问题；但由于 TLB 这类硬件和操作系统特性，访问内存中的某个页可能代价高昂，特别是当这个页当前没有被 TLB 映射时。因此，最好时刻记住这条实现上的提示：**RAM 不总是 RAM**。有时随机访问（random access）地址空间，尤其是访问的页数超过了 TLB 的覆盖范围时，可能会造成严重的性能损失。由于我们的一位导师 David Culler 以前总说 TLB 是许多性能问题的根源，我们以他的名字命名这条定律：**Culler 定律**（Culler's Law）。

mask）字段（图中没有画出），用于支持多种页大小；后面我们会看到为什么更大的页可能有用。最后，64 位中有一些位没有使用（图中灰色部分）。

MIPS 的 TLB 通常有 32 或 64 个这样的项，其中大多数由运行中的用户进程使用。不过，也有少数几项是留给操作系统的。操作系统可以设置一个 *wired* 寄存器，告诉硬件要为操作系统保留多少个 TLB 槽位；操作系统把这些保留的映射留给它在关键时刻要访问的代码和数据；在这些时刻，一旦发生 TLB 未命中就会出问题（例如在 TLB 未命中处理程序中）。

由于 MIPS 的 TLB 是由软件管理的，就得有用来更新 TLB 的指令。MIPS 提供了四条这样的指令：TLBP，探查 TLB，看某个转换是否在其中；TLBR，把一个 TLB 项的内容读到寄存器中；TLBWI，替换指定的 TLB 项；以及 TLBWR，替换一个随机的 TLB 项。操作系统用这些指令来管理 TLB 的内容。当然，这些指令必须是**特权**指令，这一点至关重要；想象一下，如果用户进程能修改 TLB 的内容，它能做出什么事来（提示：几乎什么都能做，包括接管机器、运行它自己的恶意“操作系统”，甚至让太阳（Sun）消失）。

19.8 小结

我们已经看到，硬件可以帮助我们加快地址转换。有了一个小巧的专用片上 TLB 作为地址转换缓存（address-translation cache），大多数内存引用有望不必访问主存中的页表就能完成。因此，在通常情况下，程序运行起来几乎和内存根本没被虚拟化时一样快；这对操作系统来说是一项了不起的成就，当然也是现代系统能够使用分页的必要前提。

然而，TLB 并不能让所有程序都一帆风顺。具体来说，如果程序在短时间内访问的页数超过了 TLB 能容纳的页数，程序就会产生大量的 TLB 未命中，从而运行得慢很多。我们把这种现象称为超出 **TLB 覆盖范围**（TLB coverage），对某些程序来说，这可能是个大问题。一种解决办法是支持更大的页，下一章会讨论：把关键的数据结构放到程序地址空间中用大页（large page）映射的区域，就可以增大 TLB 的有效覆盖范围。**数据库管理系统**

 © 2008–23, ARPACI-DUSSEAU

（database management system，**DBMS**）这类程序经常利用大页（large page）支持，因为它们有一些既庞大、又被随机访问的数据结构。

还有一个 TLB 问题值得一提：TLB 访问很容易成为 CPU 流水线中的瓶颈，在使用所谓**物理索引缓存**（physically-indexed cache）时尤其如此。对于这种缓存，地址转换必须在访问缓存之前完成，这可能会明显拖慢速度。由于存在这个潜在的问题，人们研究了各种巧妙的方法，用虚拟地址来访问缓存，从而在缓存命中（cache hit）时省去代价高昂的转换步骤。这种**虚拟索引缓存**（virtually-indexed cache）解决了一些性能问题，但也给硬件设计带来了新问题。更多细节请参阅 Wiggins 出色的综述 [W03]。

 © 2008–23, ARPACI-DUSSEAU

参考文献

[BC91] “Performance from Architecture: Comparing a RISC and a CISC with Similar Hardware Organization” by D. Bhandarkar and Douglas W. Clark. Communications of the ACM, September 1991. *A great and fair comparison between RISC and CISC. The bottom line: on similar hardware, RISC was about a factor of three better in performance.*

[CM00] “The evolution of RISC technology at IBM” by John Cocke, V. Markstein. IBM Journal of Research and Development, 44:1/2. *A summary of the ideas and work behind the IBM 801, which many consider the first true RISC microprocessor.*

[C95] “The Core of the Black Canyon Computer Corporation” by John Couleur. IEEE Annals of History of Computing, 17:4, 1995. *In this fascinating historical note, Couleur talks about how he invented the TLB in 1964 while working for GE, and the fortuitous collaboration that thus ensued with the Project MAC folks at MIT.*

[CG68] “Shared-access Data Processing System” by John F. Couleur, Edward L. Glaser. Patent 3412382, November 1968. *The patent that contains the idea for an associative memory to store address translations. The idea, according to Couleur, came in 1964.*

[CP78] “The architecture of the IBM System/370” by R.P. Case, A. Padegs. Communications of the ACM. 21:1, 73-96, January 1978. *Perhaps the first paper to use the term* **translation lookaside buffer**. *The name arises from the historical name for a cache, which was a* **lookaside buffer** *as called by those developing the Atlas system at the University of Manchester; a cache of address translations thus became a* **translation lookaside buffer**. *Even though the term lookaside buffer fell out of favor, TLB seems to have stuck, for whatever reason.*

[H93] “MIPS R4000 Microprocessor User’s Manual”. by Joe Heinrich. Prentice-Hall, June 1993. Available: http://cag.csail.mit.edu/raw/documents/R4400_Uman_book_Ed2.pdf *A manual, one that is surprisingly readable. Or is it?*

[HP06] “Computer Architecture: A Quantitative Approach” by John Hennessy and David Patterson. Morgan-Kaufmann, 2006. *A great book about computer architecture. We have a particular attachment to the classic first edition.*

[I09] “Intel 64 and IA-32 Architectures Software Developer’s Manuals” by Intel, 2009. Available: http://www.intel.com/products/processor/manuals. *In particular, pay attention to “Volume 3A: System Programming Guide” Part 1 and “Volume 3B: System Programming Guide Part 2”.*

[PS81] “RISC-I: A Reduced Instruction Set VLSI Computer” by D.A. Patterson and C.H. Sequin. ISCA ’81, Minneapolis, May 1981. *The paper that introduced the term RISC, and started the avalanche of research into simplifying computer chips for performance.*

[SB92] “CPU Performance Evaluation and Execution Time Prediction Using Narrow Spectrum Benchmarking” by Rafael H. Saavedra-Barrera. EECS Department, University of California, Berkeley. Technical Report No. UCB/CSD-92-684, February 1992.. *A great dissertation about how to predict execution time of applications by breaking them down into constituent pieces and knowing the cost of each piece. Probably the most interesting part that comes out of this work is the tool to measure details of the cache hierarchy (described in Chapter 5). Make sure to check out the wonderful diagrams therein.*

[W03] “A Survey on the Interaction Between Caching, Translation and Protection” by Adam Wiggins. University of New South Wales TR UNSW-CSE-TR-0321, August, 2003. *An excellent survey of how TLBs interact with other parts of the CPU pipeline, namely hardware caches.*

[WG00] “The SPARC Architecture Manual: Version 9” by David L. Weaver and Tom Germond. SPARC International, San Jose, California, September 2000. Available: www.sparc.org/standards/SPARCV9.pdf. *Another manual. I bet you were hoping for a more fun citation to end this chapter.*

 © 2008–23, ARPACI-DUSSEAU

作业（测量）

在这个作业中，你要测量 TLB 的大小以及访问 TLB 的代价。这个想法基于 Saavedra-Barrera 的工作 [SB92], 他提出了一种简单而优美的方法, 只用一个非常简单的用户级程序, 就能测量缓存层次结构的许多方面。详情可以读他的论文。

基本思路是访问一个大型数据结构（例如数组）中的若干个页，并为这些访问计时。例如，假设某台机器的 TLB 大小恰好是 4（这非常小，但便于讨论）。如果你写一个程序，只访问 4 个或更少的页，每次访问都应该是 TLB 命中，因此相对较快。然而，一旦在循环中反复访问 5 个或更多的页，每次访问的代价就会突然跳升到 TLB 未命中的水平。

遍历一次数组的基本代码大致如下：

```
int jump = PAGESIZE / sizeof(int);
for (i = 0; i < NUMPAGES * jump; i += jump)
    a[i] += 1;
```

这个循环会更新数组 a 每一页中的一个整数，总共更新 NUMPAGES 个页。把这样的循环反复运行并计时（比如在外面再套一层循环，重复几亿次，或者重复到足够运行几秒钟为止），就能算出每次访问（平均）需要多长时间。随着 NUMPAGES 的增加，观察代价在哪里出现跳变，就可以大致确定第一级 TLB 有多大，判断是否存在第二级 TLB（如果存在，它有多大），并大致了解 TLB 命中和未命中会如何影响性能。

图 19.5（第 13 页）显示了随着循环中访问的页数增加，每次访问的平均时间如何变化。从图中可以看到，只访问少数几个（8 个或更少）页时，平均访问时间大约是 5 纳秒。访问 16 个或更多的页时，每次访问的时间会突然跳到大约 20 纳秒。最后一次代价跳变出现在大约 1024 个页时，此时每次访问大约需要 70 纳秒。从这些数据可以得出结论：TLB 分为两级；第一级很小（大概能容纳 8 到 16 项）；第二级更大但更慢（大约能容纳 512 项）。第一级 TLB 命中和 TLB 未命中的代价总体相差很大，大约有 14 倍。TLB 性能很重要！

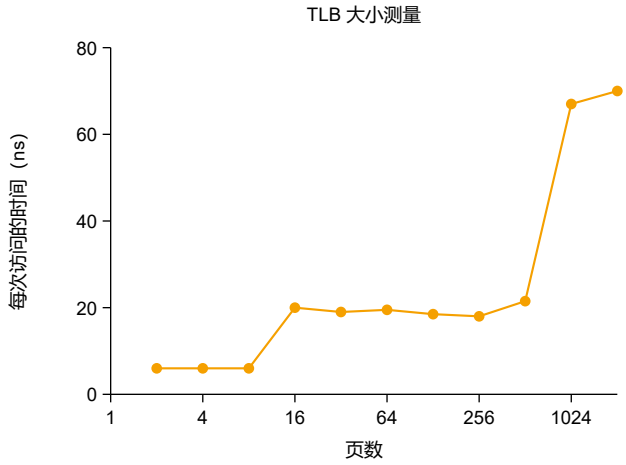


图 19.5：**测出 TLB 的大小和未命中代价**

问题

1. 计时需要用到计时器（例如 `gettimeofday()`）。这种计时器的精度如何？一个操作要持续多长时间，你才能精确地为它计时？（这有助于确定你需要在循环中把页访问重复多少次，才能成功计时。）
2. 编写一个名为 `tlb.c` 的程序，大致测量访问每个页的代价。程序的输入应该是：要访问的页数和试验次数。
3. 现在用你喜欢的脚本语言（bash?）写一个脚本来运行这个程序，让访问的页数从 1 逐步增加到几千，比如每次翻一倍。在不同的机器上运行这个脚本，收集一些数据。需要多少次试验才能得到可靠的测量结果？
4. 接下来，把结果画成图，样子和上面那张差不多。用个好用的工具，比如 ploticus，甚至 zplot。可视化通常能让数据容易理解得多；你觉得这是为什么？
5. 需要注意的一点是编译器优化。编译器会做各种巧妙的事情，比如某个循环递增的值如果之后再也没有被程序的其他部分用到，编译器就会把这个循环删掉。你怎样才能确保编译器不会把上面的主循环从你的 TLB 大小估算程序中删掉？
6. 另一点需要注意的是，如今大多数系统都配有多个 CPU，当然，每个 CPU 都有自己的 TLB 层次结构。要想真正得到好的测量结果，你必须让代码只在一个 CPU 上运行，而不是让调度程序（scheduler）把它在 CPU 之间来回迁移（migration）。怎样才能做到这一点？（提示：在 Google 上搜索“pinning a thread”找些线索。）如果不这样做，代码从一个 CPU 移到另一个 CPU 上，会发生什么？
7. 另一个可能出现的问题与初始化有关。如果在访问上面的数组 `a` 之前没有初始化它，第一次访问的代价会非常高，因为有按需置零（demand zeroing）这类初次访问的开销。这会影响你的代码及其计时吗？你能做些什么来抵消这些潜在的开销？

 © 2008–23, ARPACI-DUSSEAU